

APP STORE

Readiness & Launch Guide

A complete, plain-English walkthrough of everything Mint needs to turn this web app into a real iPhone app and get it approved on the Apple App Store. Written so anyone can follow it - what we have, what we are missing, and the exact steps to launch.

Document	Mint - App Store Readiness & Launch Guide
App name	Mint (Bundle ID: com.algohive.mint.app)
App version	1.0.0
Platform	iOS (Apple App Store) - built with Capacitor
Prepared	June 2026
Audience	Mint team - non-technical friendly

. What's inside this guide

1. The big picture – what we're doing in one minute
- 2. Plain-English dictionary (no jargon left behind)
- 3. What we ALREADY have
- 4. What we're MISSING (the rejection blockers)
- 5. Detailed breakdown - every requirement explained
- 6. Accounts, tools & money you need first
- 7. How a mobile app is built from this web app
- 8. Step-by-step: building the iPhone app
- 9. Step-by-step: submitting to the App Store
- 10. The App Store listing checklist
- 11. Money & finance app rules (important for Mint)
- 12. Full readiness checklist (one-page summary)
- 13. Recommended order of work

1. The big picture - what we're doing

Right now, Mint is a website (a web app). It runs in a browser. The goal is to wrap that same web app inside a real iPhone app and publish it on the Apple App Store so people can download it like any other app.

The good news: we are NOT starting from scratch. Mint already uses a tool called Capacitor, which takes our existing website and puts it inside a native iPhone (and Android) shell. So most of the heavy lifting is done. What's left is fixing a handful of things Apple insists on, deploying our backend to a permanent home, and going through Apple's submission process.

In one sentence

We take the Mint web app, package it as an iPhone app with Capacitor, fix 4 things Apple will otherwise reject us for, then submit it through Apple's review - which usually takes 1 to 3 days.

How an app gets onto the App Store (the journey)

1) Build the app on a Mac -> 2) Sign it with Apple certificates -> 3) Upload it to Apple -> 4) Fill in the store listing (screenshots, description, privacy) -> 5) Apple reviews it -> 6) It goes live. This guide walks through every one of these steps.

2. Plain-English dictionary

Whenever a word in this guide sounds technical, look it up here first.

Term	What it actually means
Web app	Our current Mint - a website that runs in a browser.
Native app	A real app installed on the phone, downloaded from the App Store.
Capacitor	The tool that wraps our website into a native iPhone/Android app. Already set up in Mint.
Xcode	Apple's free program (Mac only) used to build and upload iPhone apps.
Mac	An Apple computer. Required - you cannot build an iPhone app on Windows or Linux.
Apple Developer Program	The paid Apple membership (\$99/year) that lets you publish apps.
App Store Connect	Apple's website where you manage your app listing, screenshots and submit for review.
Bundle ID	Your app's unique name in Apple's system. Mint's is com.algohive.mint.app.
Certificate / Provisioning	Digital ID cards from Apple that prove the app is really from you.
Info.plist	A settings file inside the iPhone app. It holds permission messages and config.
Permission string	The sentence shown when the app asks to use the camera, Face ID, etc.
Backend	The server side of Mint - the part that talks to the database and other services.
HTTPS	A secure (encrypted) web address starting with https://. Apple requires it.
KYC	'Know Your Customer' - the identity check users do during onboarding (Sumsub).
TestFlight	Apple's tool for sending the app to testers before it goes public.

3. What we ALREADY have

Good foundations are in place. Here is what's confirmed working in the project today:

HAVE

Capacitor is installed and set up

The core tool plus the iOS and Android projects already exist in the codebase (ios/ and android/ folders).

HAVE

iPhone project structure

ios/App contains the Xcode project, app delegate, launch screen and asset folders - ready to open on a Mac.

HAVE

Face ID permission message

Info.plist already has NSFaceIDUsageDescription: 'Enable biometrics to securely sign in to Mint.'

HAVE

Native biometrics & haptics

Plugins installed: @capgo/capacitor-native-biometric (Face ID / fingerprint) and @capacitor/haptics (vibration feedback).

HAVE

App icon (1024x1024)

A high-resolution app icon is present in the iOS asset catalog.

HAVE

Splash / launch screens

Light and dark launch images exist for all required sizes.

HAVE

App identity set

App name 'Mint' and Bundle ID 'com.algohive.mint.app' are configured.

HAVE

Compliant payment model

Mint moves real money (investments, deposits, credit). These are EXEMPT from Apple's In-App Purchase fees - no IAP needed.

HAVE

Email/password + OTP login

Because there is no Google/Facebook login, Apple does NOT force us to add 'Sign in with Apple'.

4. What we're MISSING - the rejection blockers

These are the items Apple will reject the app for if they are not fixed. There are four. Each one on its own is enough to get a rejection, so all four must be done before submitting.

MISSING

1) The app loads a temporary dev website

capacitor.config.json points to a temporary Replit dev link with 'cleartext' (insecure) turned on. The app must instead run its own built-in code and talk to a permanent, secure HTTPS backend. A thin app that just opens a remote website can also be rejected for 'not enough functionality'.

MISSING

2) No 'Delete Account' feature

Apple REQUIRES any app with sign-up to let users delete their account from inside the app. Mint currently has no delete-account flow anywhere. This is an automatic rejection.

MISSING

3) Camera & microphone permission messages

KYC (Sumsu) and bank linking (TruID) use the camera and microphone to scan IDs and faces. Info.plist is missing NSCameraUsageDescription and NSMicrophoneUsageDescription. Without them the app CRASHES when KYC opens - guaranteed rejection.

MISSING

4) Privacy Policy & Terms not wired up

The in-app Legal page shows 'Privacy Policy / Terms / Cookie' buttons, but they have no action and no content behind them. Apple needs a working, publicly reachable Privacy Policy - both as a web link and inside the app.

Bottom line

Fix these four, deploy the backend to a permanent HTTPS address, and the app is technically ready to submit. Everything else is Apple paperwork and the store listing.

5. Detailed breakdown - every requirement explained

5.1 Fix the app configuration (capacitor.config.json)

Today the app is told to load a temporary web address. For a real release we change it so the app carries its own built code (the 'dist' folder) and only calls our backend for data. We also remove the 'cleartext' setting, because Apple requires secure HTTPS connections only.

- Remove the temporary 'server.url' that points to the Replit dev link.
- Remove 'cleartext: true' (insecure traffic is not allowed by Apple).
- Make sure the app's data calls go to a permanent production domain over HTTPS.

5.2 Add the missing permission messages

iPhones make apps explain WHY they need the camera, microphone, etc. These are short sentences shown in a popup. We already have the Face ID one; we need to add two more:

Permission	Why Mint needs it	Status
NSFaceIDUsageDescription	Biometric login (Face ID)	Already added
NSCameraUsageDescription	Scan ID documents & face during KYC (Sumsu) and bank linking	Must add
NSMicrophoneUsageDescription	Liveness/voice check during KYC verification	Must add

5.3 Build the 'Delete Account' feature

Users must be able to permanently delete their account from inside the app, without emailing support. A typical flow: Settings -> Delete Account -> confirm (and re-enter password) -> the account and its personal data are removed or scheduled for removal. Note: because Mint is a regulated financial app, some records may need to be kept for legal reasons - the app should explain this clearly to the user.

Heads up - this touches the backend

Deleting an account safely needs server-side work (removing the user from authentication and the database). This is in the protected server area, so it needs your explicit go-ahead before we build it.

5.4 Wire up Privacy Policy, Terms & Cookies

The Legal page buttons need to actually open real, readable documents - and the Privacy Policy must also live at a public web link (Apple asks for that link when we submit). We can either show the text inside the app or open a hosted web page.

5.5 Small but important polish

- App icon must have NO transparency and NO rounded corners (Apple adds the rounding). Ours is a solid 1024x1024, which is correct - just confirm no transparency.
- Add 'ITSAAppUsesNonExemptEncryption = false' to Info.plist so we are not asked export-compliance questions on every upload (we only use standard HTTPS).
- Confirm the app only supports portrait orientation (already set) and looks correct on modern iPhone screens with notches.

6. Accounts, tools & money you need first

Before any submission, these need to be in place. None of this is code - it's setup and paperwork.

What	Why	Cost / Notes
Apple Developer Program	Required to publish any app on the App Store.	\$99 / year
Organization (not personal) account	Mint is a financial company; finance apps should be published by the company. Apple is strict here.	Needs a D-U-N-S number (free, ~1-2 weeks to get)
A Mac computer	iPhone apps can ONLY be built and uploaded from a Mac (with Xcode).	Mac mini, MacBook, or a cloud Mac service
Xcode	Apple's free build tool. Lives on the Mac.	Free from the Mac App Store
Certificates & profiles	Digital IDs that prove the app is from Mint.	Created free inside Xcode / Apple Developer
A permanent backend host	So the app has a stable, secure place to get its data.	A production deployment over HTTPS
Public Privacy Policy URL	Apple requires a reachable privacy policy link.	A simple hosted web page

Why an Organization account matters for Mint

Apple Guideline 3.2.1 says banking and financial-services apps should be submitted by the licensed financial institution. Using a personal account for a money app risks rejection. Get the D-U-N-S number early - it can take a couple of weeks.

7. How a mobile app is built from this web app

Here is the simple mental model. Capacitor takes the finished Mint website and places it inside a native iPhone container. The container is a real app; inside it, our familiar Mint screens run. Native features (Face ID, camera, haptics) are reached through Capacitor 'plugins' we already have.

Layer	What it is	In Mint today
Web app (UI)	All the Mint screens - React code	Done - this is the existing app
Build output ('dist')	The packaged website Capacitor ships	Created by 'npm run build'
Capacitor shell	The native iPhone wrapper	Installed (ios/ folder exists)
Native plugins	Bridges to Face ID, camera, haptics	Biometric + haptics installed
Backend	Server + database (Supabase, APIs)	Exists - needs permanent hosting

8. Step-by-step: building the iPhone app

This is what happens on the Mac, in order. (We handle the code fixes first; these are the build steps.)

- 1 Apply the code fixes**
Complete the four blockers from Section 4 and the polish in Section 5 inside the project.
- 2 Build the web app**
Run the build so the latest Mint is packaged into the 'dist' folder.
- 3 Copy it into the iPhone app**
Capacitor syncs the built web app into the iOS project (the 'sync' step).
- 4 Open the project in Xcode**
On the Mac, open the iOS workspace. This is where signing and uploading happen.
- 5 Sign the app**
Sign in with the Apple Developer account and let Xcode create the certificates and profile.
- 6 Set version & build number**
Confirm the version (e.g. 1.0.0) and a build number (e.g. 1). Each upload needs a new build number.
- 7 Run it on a real iPhone**
Test the whole flow on a physical device: sign up, KYC (camera!), Face ID, investing, delete account.
- 8 Archive and upload**
Create an 'Archive' in Xcode and upload it to App Store Connect.

9. Step-by-step: submitting to the App Store

- 1 Create the app in App Store Connect**
Register the app using the Bundle ID com.algohive.mint.app and the name 'Mint'.
- 2 Test via TestFlight (recommended)**
Send the uploaded build to internal testers first to catch issues before the public sees it.
- 3 Fill in the App Privacy details**
Honestly declare what data Mint collects: identity (KYC), financial info, contact details, usage data.
- 4 Add the store listing**
App description, keywords, category (Finance), support URL, marketing URL, and the Privacy Policy link.
- 5 Upload screenshots**
Provide screenshots for the required iPhone sizes (Apple lists the exact sizes).
- 6 Provide a reviewer demo account**
Apple reviewers must log in and get PAST onboarding/KYC. Give them a working test login and any bypass needed, in the review notes.
- 7 Answer export compliance**
Confirm we only use standard encryption (HTTPS). The Info.plist flag from 5.5 makes this automatic.
- 8 Submit for review**
Send it to Apple. Review typically takes 1-3 days. They may approve, or reply with items to fix.
- 9 Release**
Once approved, release immediately or on a chosen date. Mint is now on the App Store.

The #1 reason finance apps get rejected

Reviewers can't get past the KYC wall to see the app. Always give a ready-to-use demo account and clear login steps in the 'App Review notes'. Without it, expect a rejection.

10. The App Store listing checklist

Gather these assets before you start the listing so submission is smooth:

- App name: Mint (and a subtitle, up to 30 characters).
- App description: what Mint does, key features, who it's for.
- Keywords: e.g. invest, savings, JSE, portfolio, family, South Africa.
- Category: Finance (primary).
- Screenshots: for the iPhone sizes Apple requires (use real Mint screens).
- App icon: 1024x1024, no transparency, no rounded corners (we have this).
- Privacy Policy URL: a public, working link.
- Support URL and contact email.
- App Privacy 'nutrition label': data types collected and how they're used.
- Age rating questionnaire (finance apps are usually 17+ depending on answers).
- Demo/reviewer account details in the review notes.

11. Money & finance app rules (important for Mint)

In-App Purchase - good news

Apple's 30% In-App Purchase fee applies to DIGITAL goods (game coins, premium content). It does NOT apply to real-world money and financial services. Mint's investments, deposits and credit are real financial services, so we can use our own payment flows (e.g. bank/Paystack) without Apple's IAP. Just be careful never to sell digital-only content through an outside payment method.

Sign in with Apple - not required

Apple only forces 'Sign in with Apple' when an app offers other social logins (Google, Facebook). Mint uses email/password and OTP only, so we are exempt. If we ever add social login later, we would then also need to add Sign in with Apple.

Regulation & trust

Finance apps get extra scrutiny. Be ready to show Mint is operated by a legitimate, authorised financial entity. Publish from the company (Organization) account, keep the Privacy Policy accurate about KYC and financial data, and make sure terms reflect the real services.

Data privacy

Mint handles very sensitive data: ID numbers, dates of birth, addresses, bank links and uploaded documents. The App Privacy section must declare all of this. Being inaccurate here is a common cause of rejection and of trouble later.

12. Full readiness checklist (one-page summary)

#	Item	Status	Type
1	Capacitor + iOS/Android projects exist	HAVE	Setup
2	App icon 1024 + splash screens	HAVE	Assets
3	Face ID permission message	HAVE	Config
4	Biometrics + haptics plugins	HAVE	Native
5	Compliant payments (no IAP needed)	HAVE	Policy
6	Email/OTP login (no Sign in with Apple needed)	HAVE	Policy
7	Remove dev URL + cleartext from config	MISSING	Blocker
8	Delete Account feature (in-app)	MISSING	Blocker
9	Camera + microphone permission messages	MISSING	Blocker
10	Privacy Policy & Terms wired up + public link	MISSING	Blocker
11	Backend deployed to permanent HTTPS host	MISSING	Infra
12	Encryption-exempt flag in Info.plist	MISSING	Polish
13	Apple Developer (Organization) account	MISSING	Account
14	Mac + Xcode to build	MISSING	Tooling
15	Screenshots + store listing text	MISSING	Listing
16	App Privacy 'nutrition label' filled in	MISSING	Listing
17	Reviewer demo account + notes	MISSING	Listing

13. Recommended order of work

If we tackle it in this order, nothing blocks anything else:

- 1 Code fixes (in this project)**
Camera/mic permissions, Privacy Policy/Terms content, Capacitor config cleanup, encryption flag. (Delete Account needs your go-ahead as it touches the backend.)
- 2 Deploy the backend**
Get the server + APIs onto a permanent, secure HTTPS production address.
- 3 Start Apple paperwork in parallel**
Apply for the Organization Apple Developer account and the D-U-N-S number now - it takes the longest.
- 4 Build & test on a Mac**
Build the iPhone app, run it on a real device, and check every flow (especially KYC camera and Delete Account).
- 5 TestFlight**
Send to internal testers, fix anything that comes up.
- 6 Submit & launch**
Complete the listing, add the reviewer account, submit, and release once approved.

What we can start right now

The code-side blockers (camera/mic permissions, privacy policy, config cleanup, encryption flag) can be done immediately inside this project. The Delete Account feature needs your OK because it touches the protected backend. Everything else is Apple account setup and the store listing.